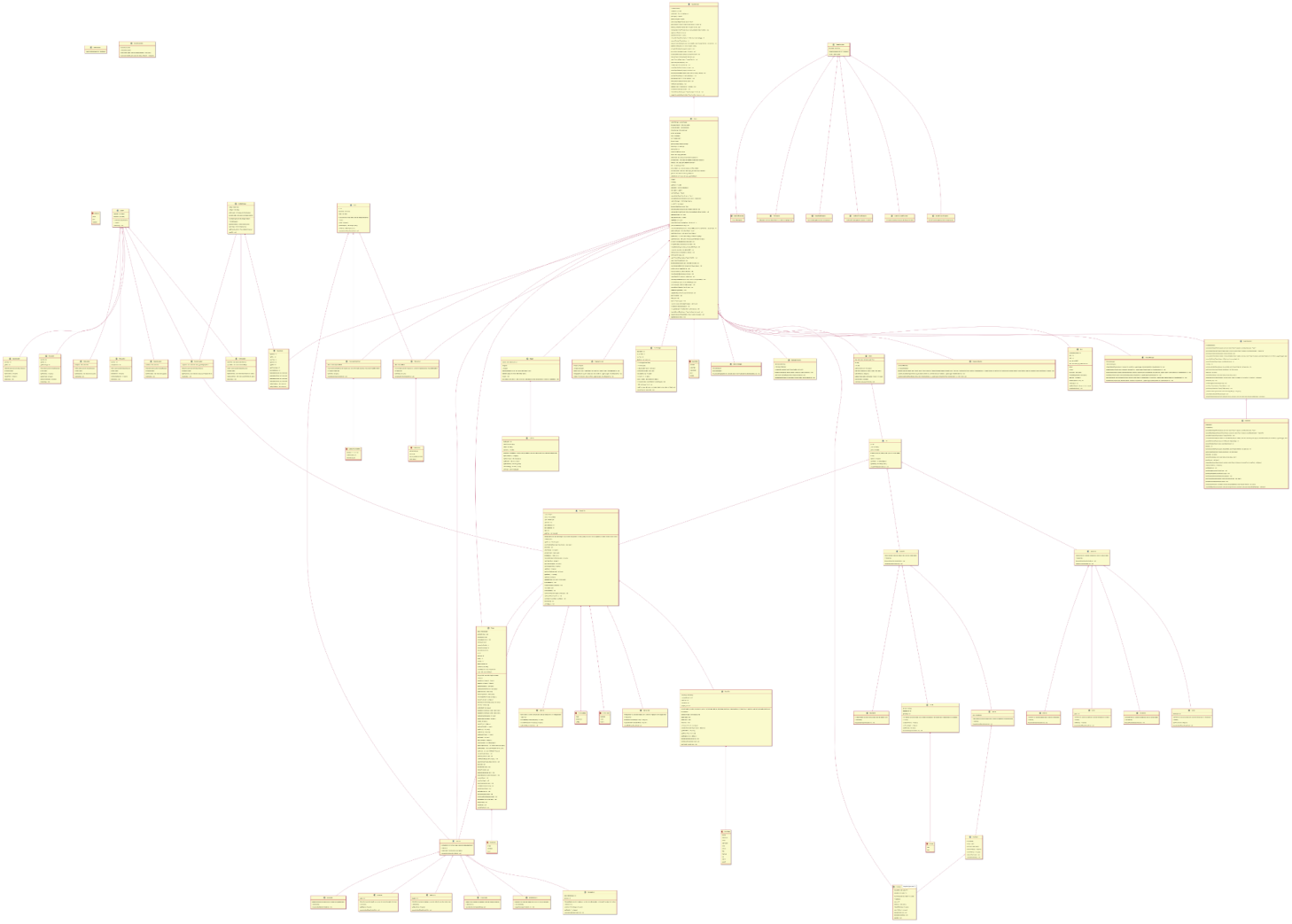
Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

Kode Kelompok : LLM
Nama Kelompok : Liro Language Model

1. 13524122 / Nathaniel Christian
2. 13524133 / M. Daffa Arrizki Yanma
3. 13524135 / Varistha Devi
4. 13524139 / Azri Arzaq Pohan
5. 13524142 / Rasyad Satyatma

LLM - Liro Language Model

1. Diagram Kelas



Gambar 1.1 Diagram Kelas [Link Diagram Kelas]

2. Deskripsi Kelas

2.1. Tile

Tile adalah abstract base class yang menjadi fondasi dari seluruh sistem petak (tile) pada papan permainan. Kelas ini mendefinisikan kontrak umum yang harus dipenuhi oleh setiap petak: setiap petak memiliki id (int), code (std::string), dan name (std::string) sebagai identitas dasarnya.

Metode kunci dari kelas ini adalah landedOn(IGameContext& ctx), yang dideklarasikan sebagai pure virtual. Hal ini berarti setiap kelas turunan wajib mendefinisikan sendiri apa yang terjadi ketika seorang pemain mendarat di petak tersebut. Dengan demikian, Tile berperan sebagai blueprint polimorfis: game loop cukup memanggil tile->landedOn(ctx) tanpa perlu tahu tipe konkret dari petak yang bersangkutan.

Tile memiliki dua cabang turunan langsung: PropertyTile (untuk petak yang dapat dimiliki) dan SpecialTile/ActionTile (untuk petak dengan efek khusus).

2.2. PropertyTile

PropertyTile merepresentasikan semua petak yang dapat dibeli, dimiliki, dan disewakan kepada pemain lain. Kelas ini mewarisi Tile dan menambahkan atribut seperti owner (Player*), status (PropertyStatus), type (PropertyType), price, mortgageValue, dan rentTable (std::vector<int>).

Tanggung jawab utama PropertyTile adalah mengelola siklus kepemilikan: dari status BANK (belum dibeli), berpindah ke OWNED setelah pemain membelinya, hingga ke MORTGAGED apabila dijaminkan. Metode landedOn() mengimplementasikan logika pembayaran sewa kepada pemilik bila properti sudah dimiliki. Untuk petak yang masih berstatus BANK, logika penawaran pembelian ditangani oleh IGameContext::promptBuyProperty() yang dipanggil dari dalam landedOn().

PropertyTile juga mengelola mekanisme festival (activateFestival, tickFestival) yang merupakan fitur bonus. Kelas ini berrelasi dengan Player melalui pointer owner dan dengan IGameContext untuk semua operasi finansial.

Kelas ini memiliki tiga kelas turunan: StreetTile, RailroadTile, dan UtilityTile, yang masing-masing mengoverride calculateRent() untuk menghitung sewa sesuai aturannya sendiri.

2.3. StreetTile

StreetTile adalah turunan dari PropertyTile yang merepresentasikan petak tanah berwarna pada papan Monopoli. Kelas ini menambahkan atribut colorGroup (ColorGroup), propertyLevel (int), housePrice, hotelPrice, dan monopolyOwned (bool).

Metode calculateRent() di StreetTile menghitung sewa berdasarkan level properti (rumah/hotel) dan status monopoli. Apabila pemilik memiliki semua petak dalam satu kelompok warna (monopoly), sewa pada properti dengan level 0 berlipat ganda. Semakin banyak rumah atau hotel yang dibangun, sewa semakin tinggi sesuai rentTable.

StreetTile juga mendukung operasi buildHouse(), buildHotel(), dan demolish() yang digunakan oleh PropertyManager. Relasi dengan ColorGroup enum memungkinkan pengecekan status monopoli dilakukan secara efisien.

2.4. RailroadTile

RailroadTile mewarisi PropertyTile dan merepresentasikan petak stasiun kereta. Sewa pada RailroadTile tidak bergantung pada pembangunan rumah, melainkan pada jumlah stasiun yang dimiliki pemilik. Metode calculateRent() memiliki dua versi overload: calculateRent(const Player& player) menghitung berdasarkan jumlah railroad yang dimiliki player tersebut, sedangkan calculateRent(IGameContext& ctx) mengambil player aktif dari context dan mendelegasikan ke versi pertama. Pola ini memberikan fleksibilitas pemanggilan dari konteks berbeda.

RentTable untuk railroad diisi dari konfigurasi eksternal via RailroadLoader dan disimpan dalam GameConfig, sehingga nilai sewa dapat dikustomisasi tanpa mengubah kode.

2.5. UtilityTile

UtilityTile mewarisi PropertyTile dan merepresentasikan petak utilitas (misalnya perusahaan air dan listrik). Seperti RailroadTile, metode calculateRent() juga memiliki dua versi overload: calculateRent(const Player& player) menghitung sewa berdasarkan jumlah utility yang dimiliki player tersebut dikalikan dengan hasil dadu terakhir (yang harus diakses dari context eksternal), sedangkan calculateRent(IGameContext& ctx) mengambil baik player pemilik maupun hasil dadu terakhir via ctx.getLastDiceTotal() lalu menghitung sewa. Pola ini memberikan fleksibilitas pemanggilan dari konteks berbeda.

Relasi penting kelas ini adalah ketergantungannya pada hasil dadu terakhir, yang harus diakses lewat IGameContext agar tidak ada coupling langsung antara Tile dengan Dice.

2.6. SpecialTile dan Turunannya

SpecialTile adalah kelas intermediate yang mewarisi Tile dan mengimplementasikan landedOn() dengan mendelegasikan ke pure virtual method executeAction(IGameContext& ctx). Pola Template Method ini memastikan semua SpecialTile diperlakukan seragam dari luar, namun masing-masing mendefinisikan aksinya sendiri di dalam.

Kelas-kelas turunan SpecialTile antara lain:

- GoTile: memberikan gaji (salary) kepada pemain yang melewati atau mendarat di petak GO via IGameContext::grantSalary().
- JailTile: menangani petak Just Visiting/In Jail. Pemain yang berkunjung tidak terkena efek; pemain yang berada di penjara harus membayar denda (fineAmount) atau melempar dadu ganda.
- GoToJailTile: secara langsung mengirim pemain ke penjara via IGameContext::sendPlayerToJail().
- FreeTile: petak parkir gratis tanpa efek apapun; executeAction() kosong.
- FestivalTile: mengaktifkan mekanisme festival pada sebuah properti yang dipilih pemain.

2.7. ActionTile dan CardTile

ActionTile mewarisi Tile dan juga mengimplementasikan pola Template Method yang serupa dengan SpecialTile: landedOn() mendelegasikan ke executeAction(). ActionTile digunakan untuk petak yang memicu efek dari deck kartu (Chance dan Community Chest).

CardTile mewarisi ActionTile dan menyimpan referensi ke ICardDeck& (bukan pointer, melainkan reference untuk menjamin deck selalu valid) dan mengimplementasikan executeAction() dengan menarik satu kartu dari deck tersebut lalu mengeksekusinya. Pemisahan antara ICardDeck (interface) dan CardDeck<T> (implementasi template) memungkinkan CardTile tidak bergantung pada tipe kartu konkret.

TaxTile adalah turunan ActionTile lainnya yang menangani dua jenis pajak: PPH (flat atau persentase dari total kekayaan) dan PBM (flat). Logika pemilihan metode pajak (flat vs. persentase) didelegasikan ke pemain via IGameContext::promptTaxChoice().

2.8. Hierarki Card

Card adalah base class untuk seluruh kartu dalam permainan, dengan atribut id, name, dan description, serta pure virtual method executeAction(IGameContext& ctx).

Dari Card, terdapat dua cabang hierarki. Pertama adalah kartu event (ChanceCard dan CommunityChestCard) yang masing-masing memiliki enum effect untuk menentukan efek yang terjadi ketika kartu ditarik dari CardTile. Kedua adalah SkillCard beserta turunannya

(ShieldCard, DiscountCard, MoveCard, LassoCard, DemolitionCard, TeleportCard) yang merupakan kartu kemampuan yang dipegang oleh pemain di tangan (hand).

CardDeck<T> adalah kelas template yang mengelola draw pile dan discard pile dari sebuah jenis kartu. Kelas ini mengimplementasikan interface ICardDeck, sehingga CardTile dapat menggunakan kartu apapun tanpa bergantung tipe konkretnya. Kartu ditarik dengan draw() dan dikembalikan ke discard pile dengan discard(). Ketika draw pile habis, reshuffleDiscardPile() mengisi ulang secara otomatis.

2.9. Player

Player merepresentasikan seorang peserta dalam permainan. Kelas ini menyimpan semua state pemain: posisi di papan (position), jumlah uang (money), status (PlayerStatus: ACTIVE/JAILED/BANKRUPT), daftar properti yang dimiliki (properties), dan kartu di tangan (hand). Kelas ini juga menyimpan flag isComputer (bool) untuk membedakan pemain manusia dan COM (Computer Player) dalam mode bonus.

Kelas Player juga menangani mekanisme jail (jailTurns, getOutOfJailCard), efek shield (shieldTurnsLeft), dan efek diskon (discountPercentage, discountTurnsLeft). Metode-metode seperti tickShield() dan tickDiscount() dipanggil di setiap akhir giliran untuk mendekrementasi durasi efek tersebut.

Player meng-overload operator += dan -= untuk operasi penambahan dan pengurangan uang yang lebih ekspresif, serta operator < dan > untuk perbandingan kekayaan antar pemain (berguna saat menentukan pemenang atau urutan lelang).

Relasi Player dengan PropertyTile bersifat bidireksional: Player menyimpan daftar PropertyTile* yang dimilikinya, sementara PropertyTile menyimpan Player* sebagai owner-nya. Manajemen relasi ini dilakukan oleh IGameContext::grantProperty().

2.10. Game dan IGameContext

Game adalah kelas utama yang mengimplementasikan interface IGameContext. Kelas ini mengkoordinasikan seluruh subsistem permainan: Board, Dice, TurnManager, Logger, serta manager-manager seperti AuctionManager, BankruptcyHandler, FinanceManager, PropertyManager, dan CommandHandler.

IGameContext adalah interface yang mengekspos operasi-operasi game ke komponen internal (Tile, Card, dan lainnya) tanpa mengekspos detail implementasi Game secara penuh. Ini penting untuk mencegah circular dependency: misalnya Tile tidak perlu #include "Game.hpp" yang besar, cukup #include "IGameContext.hpp". Semua operasi seperti movePlayerBy(), chargeToBank(), grantProperty(), dan sendPlayerToJail() didefinisikan di interface ini.

Game menyimpan Board sebagai std::unique_ptr<Board> dan semua pemain sebagai std::vector<std::unique_ptr<Player>>, sehingga Game bertanggung jawab atas lifetime dari objek-objek tersebut (ownership semantics yang jelas).

2.11. Board dan BoardFactory

Board menyimpan seluruh petak papan dalam std::vector<std::unique_ptr<Tile>> dan menyediakan metode akses seperti getTileAt(int), getTileByCode(const std::string&), dan getNewPosition(int, int) yang menangani wrapping posisi modulo jumlah total petak.

BoardFactory adalah factory class yang bertanggung jawab membangun objek Board lengkap dari konfigurasi yang dibaca oleh ConfigManager. Pemisahan proses konstruksi ini dari Game membuat kode lebih modular dan memudahkan pengujian Board secara terpisah.

2.12. Kelas-Kelas Manager

Desain membagi tanggung jawab Game ke beberapa kelas manager yang masing-masing memiliki domain spesifik:

- FinanceManager: mengelola semua transaksi uang (chargeToBank, transferMoney, grantSalary, collectFromAll, payToAll) dengan memeriksa solvabilitas pemain dan memicu kebangkrutan bila perlu.
- BankruptcyHandler: menangani proses kebangkrutan pemain, termasuk liquidasi aset (computeMaxLiquidation), transfer properti ke kreditur (transferAssetsToPlayer), atau pengembalian aset ke bank (returnAssetsToBank).
- AuctionManager: menjalankan mekanisme lelang properti (runAuction) dengan mengelola putaran penawaran antar pemain aktif.
- PropertyManager: mengelola operasi build, mortgage, dan redeem properti melalui interaksi dengan pemain dan Board.
- CommandHandler: mem-parsing dan mendispatch perintah teks dari pemain (seperti "lempar dadu", "beli", "jual", "gunakan kemampuan") ke handler yang sesuai.

2.13. Sistem Konfigurasi (Loader Classes)

Konfigurasi permainan dibaca dari file eksternal oleh serangkaian Loader class yang semuanya mewarisi kelas Loader. Loader menyimpan fileBuffer dan filename, serta mendefinisikan interface loadConfig() yang dipanggil oleh setiap kelas turunan.

Turunan-turunan Loader antara lain: PropertyLoader (memuat data PropertyTile), ActionLoader (memuat ActionTile dan SpecialTile), RailroadLoader (memuat rent table railroad), UtilityLoader (memuat multiplier utility), TaxLoader (memuat konfigurasi pajak), SpecialLoader (memuat salary GO dan denda jail), serta MiscLoader (memuat maxTurn dan startingBalance).

Semua loader dikoordinasikan oleh ConfigManager, yang menyimpannya dan mengeksposnya ke BoardFactory dan komponen lain yang membutuhkan data konfigurasi. Pola ini memisahkan parsing konfigurasi dari logika game, sehingga format file konfigurasi dapat diubah tanpa mengubah logika permainan.

2.14. Logger dan TurnManager

Logger menyimpan semua kejadian selama permainan sebagai std::vector<LogEntry>. Setiap LogEntry mencatat turnNumber, username, actionType, dan details. Logger menyediakan getRecentLogs() untuk ditampilkan di layar dan getSaveLogs() untuk disimpan ke file save.

TurnManager mengelola urutan giliran pemain (turnOrder sebagai std::vector<int>), indeks pemain aktif (activeIndex), dan nomor giliran saat ini (currentTurn). Metode advance() dipanggil setiap kali giliran berakhir untuk berpindah ke pemain aktif berikutnya, melewati pemain yang sudah bankrupt. removePlayer() digunakan saat pemain dieliminasi dari permainan.

2.15. IUserInteraction dan GameView

IUserInteraction adalah interface yang memisahkan logika game dari cara input/output dilakukan. Semua prompt kepada pemain (promptBuyProperty, promptAuctionBid, promptTaxChoice, readLine, dsb.) didefinisikan sebagai pure virtual di interface ini.

GameView adalah implementasi konkret IUserInteraction untuk antarmuka berbasis teks (CLI). Dengan adanya interface ini, pengujian otomatis dapat dilakukan menggunakan mock implementation, dan di masa depan GUI dapat dibuat tanpa mengubah logika game.

2.16. Hierarki Exception

Proyek mendefinisikan beberapa kelas exception khusus: GameException (base class), InsufficientFundsException, InvalidCommandException, InvalidPropertyException, GameStateException, FileException, dan CardLimitException. Setiap exception membawa informasi yang relevan dengan kondisi error yang terjadi, memungkinkan penanganan error yang granular di lapisan yang tepat.

2.17. GameSaveLoader

GameSaveLoader adalah kelas yang bertanggung jawab atas serialisasi dan deserialisasi state permainan dari/ke file eksternal. Kelas ini memisahkan logika save/load dari kelas Game, mengikuti prinsip Single Responsibility.

Metode save(const Game& game, const std::string& filename) mengekstrak seluruh state game (pemain, properti, deck, turn manager, logger) dan menuliskannya ke file teks terstruktur sesuai format yang ditentukan spesifikasi. Metode load(Game& game, const std::string& filename) melakukan operasi sebaliknya: membaca file dan merekonstruksi state game.

Pemisahan ini membuat Game tidak perlu tahu detail format file save, dan memungkinkan perubahan format penyimpanan tanpa mengubah logika Game itu sendiri.

2.18. Dice

Dice mengelola mekanisme pelemparan dua dadu d6. Kelas ini menyimpan state dadu terakhir (die1, die2) dan menghitung consecutive doubles (consecutiveDoubles) untuk deteksi speeding violation (3x double berturut-turut).

Perbedaan signifikan dari desain M1 adalah penggunaan C++ <random> library: atribut rng (std::mt19937) dan dist (std::uniform_int_distribution<int>) menggantikan std::rand() yang tidak thread-safe dan kurang random. Metode rollRandom() menghasilkan nilai dadu yang lebih fair, sementara rollManual(int d1, int d2) tetap ada untuk keperluan testing dan debugging.

Metode isDouble() mengecek apakah kedua dadu bernilai sama, dan isSpeedingViolation() mengecek apakah pemain telah melempar double tiga kali berturut-turut dalam satu giliran.

2.19. Enum-Enum Pendukung

Sistem menggunakan beberapa enumerate untuk merepresentasikan state dan kategori dalam permainan, seperti:

- ColorGroup: mengelompokkan StreetTile (BROWN, LIGHT_BLUE, PINK, ORANGE, RED, YELLOW, GREEN, DARK_BLUE) serta tipe khusus (RAILROAD, UTILITY). Enum ini digunakan untuk pengecekan monopoli dan validasi build.
- PropertyStatus: merepresentasikan status kepemilikan properti (BANK = belum dibeli, OWNED = dimiliki pemain, MORTGAGED = digadaikan).
- PlayerStatus: merepresentasikan kondisi pemain (ACTIVE = sedang bermain, JAILED = di penjara, BANKRUPT = sudah keluar dari permainan).
- GameState: mengontrol fase permainan (MENU, PLAYING, AUCTION, LIQUIDATION, GAMEOVER). Transisi antar state dikelola oleh Game dan dicek oleh CommandHandler untuk validasi perintah.
- ChanceEffect dan CommunityChestEffect: enum yang menentukan efek spesifik dari kartu Chance dan Community Chest, memisahkan data (jenis efek) dari behavior (implementasi di executeAction()).

3. Justifikasi Desain

3.1 Alasan Pemilihan Desain Secara Umum

Desain kelas pada proyek Nimonspoli ini dibangun di atas prinsip Single Responsibility Principle (SRP) dan Dependency Inversion Principle (DIP) dari SOLID OOP. Setiap kelas memiliki satu alasan untuk berubah: Tile hanya berubah bila kontrak petak berubah, FinanceManager hanya berubah bila logika finansial berubah, dan seterusnya.

Penggunaan interface IGameContext dan IUserInteraction adalah keputusan desain paling signifikan. Dengan memisahkan kontrak dari implementasi, semua Tile dan Card dapat beroperasi tanpa bergantung langsung pada kelas Game yang besar. Ini menghilangkan circular dependency antara komponen dan membuat setiap kelas dapat diuji secara independen dengan mock object.

Pemilihan inheritance untuk hierarki Tile (Tile → SpecialTile/ActionTile → kelas konkret) tepat karena papan Monopoli memang memiliki taksonomi petak yang bersifat "is-a": GoTile memang adalah SpecialTile, StreetTile memang adalah PropertyTile. Polimorfisme melalui virtual landedOn() memungkinkan game loop yang bersih tanpa switch-case besar.

Template digunakan pada CardDeck<T> karena deck Chance, Community Chest, dan SkillCard memiliki logika identik (draw, discard, reshuffle), tetapi bertipe berbeda. Penggunaan template menghilangkan duplikasi kode tanpa mengorbankan type safety.

3.2 Perubahan Desain dari Milestone 1

Dari Milestone 1 ke Milestone 2, terdapat beberapa perubahan desain yang signifikan:

- Pemisahan Game menjadi Manager Classes: Pada M1, banyak logika yang terpusat di kelas Game, sehingga kelas tersebut terlalu besar (God Object). Di M2, logika finansial dipindah ke FinanceManager, logika kebangkrutan ke BankruptcyHandler, logika lelang ke AuctionManager, dan logika properti ke PropertyManager. Ini membuat Game menjadi koordinator, bukan executor.
- Penambahan IUserInteraction: Pada M1, interaksi dengan pengguna dilakukan langsung di dalam logika game. Di M2, interface IUserInteraction diekstrak dan GameView dibuat sebagai implementasinya, memisahkan presentasi dari logika.
- Penambahan SkillCard hierarchy: Mekanisme kartu kemampuan (SkillCard beserta ShieldCard, DiscountCard, MoveCard, LassoCard, DemolitionCard, TeleportCard) ditambahkan di M2 sebagai fitur bonus kreasi mandiri.

- Refactor sistem konfigurasi: Pada M1, nilai konfigurasi masih hardcoded atau diparse di satu tempat. Di M2, sistem Loader yang terspesialisasi (RailroadLoader, TaxLoader, SpecialLoader, dll.) dan dikoordinasikan oleh ConfigManager memisahkan tanggung jawab parsing dengan jelas.
- Penambahan interface ICardDeck: Pada M1, CardTile bergantung langsung pada CardDeck<T> template class, yang mengakibatkan coupling kuat dengan tipe kartu konkret. Di M2, interface ICardDeck diekstrak sehingga CardTile dapat bekerja dengan deck apapun tanpa mengetahui tipe kartu di dalamnya. Ini memungkinkan polymorphic behavior pada level deck.

3.3 Kelebihan Desain

- Extensible: Menambahkan jenis petak baru hanya membutuhkan pembuatan subclass baru dari Tile tanpa mengubah kode yang sudah ada (Open/Closed Principle).
- Testable: Karena Game bergantung pada interface IGameContext dan IUserInteraction, setiap komponen dapat diuji dengan mock tanpa membutuhkan state game penuh.
- Readable: Pola Template Method pada SpecialTile dan ActionTile membuat alur eksekusi mudah dipahami. Siapapun yang membaca kode tahu bahwa landedOn() selalu menuju executeAction().
- Konfigurasi fleksibel: Nilai-nilai seperti harga properti, sewa, dan aturan pajak dibaca dari file eksternal melalui sistem Loader, sehingga game dapat dikustomisasi tanpa recompile.
- Type-safe namun fleksibel: Penggunaan template pada CardDeck<T> memberikan type safety compile-time (tidak bisa mencampur ChanceCard dengan SkillCard dalam satu deck), namun interface ICardDeck memungkinkan runtime polymorphism saat diperlukan.

3.4 Kekurangan Desain

- Kompleksitas awal tinggi: Jumlah kelas yang besar (40+ kelas) membuat onboarding anggota baru lebih sulit. Pemahaman terhadap IGameContext sebagai pusat ketergantungan perlu didapat lebih dahulu sebelum dapat berkontribusi.
- Overhead interface: Penggunaan interface IGameContext dan IUserInteraction menambah indirection. Debugging perlu melacak call melalui lapisan interface sebelum sampai ke implementasi konkret.
- Relasi bidireksional Player-PropertyTile: Player menyimpan vector<PropertyTile*> dan PropertyTile menyimpan Player*, yang berarti ada coupling dua arah. Ini membutuhkan kehati-hatian dalam penghapusan/perpindahan kepemilikan agar tidak terjadi dangling pointer.

- Fragmentasi konfigurasi: Adanya 6+ kelas Loader berbeda (PropertyLoader, RailroadLoader, UtilityLoader, TaxLoader, SpecialLoader, ActionLoader, MiscLoader) membuat sistem konfigurasi terfragmentasi. Meskipun ConfigManager mengkoordinasikan semuanya, menambahkan field konfigurasi baru memerlukan modifikasi di banyak tempat.

3.5 Kendala dalam Pemilihan Desain

Tantangan terbesar dalam desain adalah menentukan batas tanggung jawab antara Tile/Card dengan Game/Manager. Di satu sisi, Tile harus dapat "mengeksekusi efeknya sendiri" (e.g., memindahkan pemain, mencharge uang), namun di sisi lain semua state permainan ada di Game. Solusinya adalah IGameContext: Tile mendapat akses ke operasi yang diperlukan tanpa memiliki referensi langsung ke Game.

Kendala lain adalah desain CardDeck<T> sebagai template. Di C++, implementasi template harus berada di header file, yang berarti semua detail implementasi CardDeck terekspos ke semua file yang meng-include-nya. Untuk menyembunyikan detail ini dari CardTile, interface ICardDeck (non-template) digunakan sebagai intermediary sehingga CardTile hanya bergantung pada interface tersebut.

Penentuan apakah menggunakan inheritance atau composition juga menjadi pertimbangan. Untuk Manager Classes (FinanceManager, BankruptcyHandler, dll.), dipilih composition (Game memiliki instance manager sebagai atribut) dibanding inheritance (Game : FinanceManager) karena satu Game perlu memiliki semua kapabilitas sekaligus, dan inheritance berganda di C++ dapat menyebabkan ambiguitas.

Kendala implementasi lainnya adalah synchronization antara std::vector<PropertyTile*> di Player dengan status kepemilikan di PropertyTile itu sendiri. Saat properti berpindah kepemilikan (misalnya dalam lelang atau kebangkrutan), kedua struktur data harus diupdate secara atomik melalui IGameContext::grantProperty() untuk mencegah inkonsistensi. Debugging bug terkait ownership sering kali memerlukan tracing melalui beberapa lapisan abstraksi.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Inheritance merupakan salah satu konsep dalam pemrograman berbasis objek yang memungkinkan suatu kelas memiliki kelas turunan yang mewarisi atribut dan method dari kelas induknya. Kelas turunan (derived class) akan membawa seluruh atribut dan method yang

dimiliki oleh kelas induk (base class), dan dapat pula menambahkan atribut serta method baru, atau menimpa (override) method yang sudah ada.

4.1.1. Tile

Konsep inheritance dan polymorphism pada kelas Tile yang mendefinisikan kontrak dasar bagi seluruh petak yang ada di papan permainan. Kelas Tile digunakan untuk mendefinisikan objek petak secara umum; setiap petak di papan Monopoli memiliki sifat dan perilaku yang berbeda ketika pemain mendarat di atasnya. Dengan menggunakan inheritance, berbagai jenis petak dapat dibentuk dengan memiliki atribut dan method yang sama melalui base class, namun dengan perilaku yang berbeda-beda melalui override method virtual. Penggunaan inheritance pada kasus ini tepat karena papan memiliki banyak tipe petak yang berbeda (GO, Street, Railroad), namun semuanya berbagi satu sifat fundamental yang sama. Alih-alih menuliskan logika if/switch yang besar di satu tempat untuk menangani setiap tipe petak, inheritance memungkinkan setiap tipe petak mendefinisikan sendiri reaksinya (landedOn), sementara game loop cukup memanggil satu method yang sama untuk semua tipe petak tanpa peduli tipe konkretnya.

```
#pragma once
#include <string>


// Forward declaration — avoids pulling in Player/Game headers here
class IGameContext;


/// @brief Abstract base class for all board tiles.
class Tile {
private:
    int id;
    std::string code;
    std::string name;


public:
    Tile(int id, const std::string& code, const std::string& name);
    virtual ~Tile();
```

```
    int getId() const;
    const std::string& getCode() const;
    const std::string& getName() const;


    /**
     * @brief Called when a player lands on this tile.
     * @param ctx Live game context — use to move players, charge money, trigger events.
     */
    virtual void landedOn(IGameContext& ctx) = 0;
};



#include "Tile.hpp"


Tile::Tile(const int id, const std::string& code, const std::string& name)
    : id(id), code(code), name(name) {}


Tile::~Tile() = default;


int Tile::getId() const {
    return id;
}


const std::string& Tile::getCode() const {
    return code;
}
```

```
const std::string& Tile::getName() const {
    return name;
}
```

Kelas Tile memiliki satu pure virtual method landedOn() yang ditandai dengan = 0. Ini menjadikan Tile sebuah abstract class yang tidak dapat diinstansiasi secara langsung, melainkan harus diturunkan, dan setiap kelas turunan wajib mengimplementasikan landedOn().

4.1.1.1. SpecialTile

SpecialTile adalah kelas intermediate yang mewarisi dari Tile dan mengimplementasikan landedOn() dengan mendelegasikan ke pure virtual method executeAction(). Pola ini memungkinkan semua SpecialTile berbagi logika tanpa duplikasi, sementara aksi spesifiknya didefinisikan di kelas turunan.

```
#pragma once
#include "Tile.hpp"

/**
 * @brief Intermediate base for corner/special tiles (Go, Jail, Free Parking, Go To
Jail).
 * Same pattern as ActionTile: landedOn() delegates to executeAction().
 */
class SpecialTile : public Tile {
public:
    SpecialTile(int id, const std::string& code, const std::string& name);
    ~SpecialTile() override;

    void landedOn(IGameContext& ctx) override;

protected:
    virtual void executeAction(IGameContext& ctx) = 0;
};
```

```
#include "SpecialTile.hpp"
#include "IGameContext.hpp"

using namespace std;

SpecialTile::SpecialTile(int id, const string& code, const string& name) : Tile(id,
code, name) {}

SpecialTile::~SpecialTile() {}

void SpecialTile::landedOn(IGameContext& ctx) {
    executeAction(ctx);
}
```

4.2. Method/Operator Overloading

Method Overloading memungkinkan beberapa method atau fungsi memiliki nama yang sama namun dengan signature yang berbeda.

4.2.1. Player

Pada kelas Player, operator += dan -= di-overload untuk merepresentasikan penambahan dan pengurangan uang. Operator < dan > dioverload pada Player untuk membandingkan pemain berdasarkan uang yang dimiliki.

```
// ── Operator overloads ──────────────────────────────────────────────────────

Player& Player::operator+=(int amount) {
    money += amount;
    return *this;
}

Player& Player::operator-=(int amount) {
    money -= amount;
```

```
    return *this;
}

bool Player::operator<(const Player& other) const {
    return money < other.money;
}

bool Player::operator>(const Player& other) const {
    return money > other.money;
```

4.2.1.1. BankruptcyHandler

Pengguaan Operator += dan -= dari Player digunakan secara langsung untuk mentransfer sisa uang

```
void BankruptcyHandler::transferAssetsToPlayer(Player& debtor, Player& creditor,
                                               IGameContext& ctx) {
    const auto debtorProperties = debtor.getProperties();
    for (PropertyTile* prop : debtorProperties) {
        const bool wasMortgaged = prop->isMortgaged();
        ctx.grantProperty(creditor, *prop);
        if (wasMortgaged) {
            prop->setStatus(PropertyStatus::MORTGAGED);
        }
    }
    creditor += debtor.getMoney();
    debtor -= debtor.getMoney();
    ctx.refreshMonopolyStatus();
}
```

4.3. Template & Generic Classes

Generic class adalah fitur yang memungkinkan berbagai jenis tipe data menjadi parameter untuk suatu kelas ataupun fungsi. Dalam C++, fitur ini diimplementasikan melalui template class. Penggunaan template meningkatkan efisiensi kode dan mengurangi repetisi untuk tipe data yang berbeda, karena satu implementasi dapat digunakan untuk berbagai tipe sekaligus dengan type safety penuh yang dijamin oleh compiler pada waktu kompilasi. Pada proyek ini, konsep template diterapkan melalui kelas CardDeck yang mengelola tumpukan kartu secara generik. Proyek ini memiliki tiga jenis deck kartu yang berbeda (ChanceCard, CommunityChestCard, dan SkillCard). Tanpa template, tiga kelas terpisah harus ditulis dengan kode yang hampir identis sepenuhnya, yang merupakan duplikasi tidak perlu. Dengan satu template CardDeck, ketiga deck dibuat dari satu implementasi yang sama.

```
#pragma once
#include <vector>

class Card;

/**
 * @brief Non-templated deck interface — lets CardTile hold any deck without
 * coupling to a specific card type.
 */
class ICardDeck {
public:
    virtual ~ICardDeck() = default;
    virtual Card* draw() = 0;
    virtual void discard(Card* card) = 0;
    virtual bool isEmpty() const = 0;
    virtual void reshuffleDiscardPile() = 0;
    virtual int drawPileSize() const = 0;
    virtual int discardPileSize() const = 0;
};

/**
 * @brief Type-safe generic deck. T must publicly derive from Card.
 * @tparam T Concrete card type stored in this deck.
 */
template <typename T>
class CardDeck : public ICardDeck {
private:
```

```
    std::vector<T*> drawPile;
    std::vector<T*> discardPile;

public:
    explicit CardDeck(const std::vector<T*>& cards);
    ~CardDeck() override;

    T* draw() override;
    void discard(Card* card) override;
    void shuffle();
    void reshuffleDiscardPile() override;

    bool isEmpty() const override;
    int drawPileSize() const override;
    int discardPileSize() const override;
};

#include "Deck.hpp"
#include "Card.hpp"
#include "ChanceCard.hpp"
#include "CommunityChestCard.hpp"
#include "SkillCard.hpp"
#include <algorithm>
#include <random>

using namespace std;

template <typename T>
CardDeck<T>::CardDeck(const vector<T*>& cards) : drawPile(cards) {
    shuffle();
}

template <typename T>
CardDeck<T>::~CardDeck() {}

template <typename T>
```

```
T* CardDeck<T>::draw() {
    if (drawPile.empty()) {
        reshuffleDiscardPile();
    }
    if (drawPile.empty()) {
        return nullptr;
    }
    T* card = drawPile.back();
    drawPile.pop_back();
    return card;
}

template <typename T>
void CardDeck<T>::discard(Card* card) {
    T* typed = dynamic_cast<T*>(card);
    if (typed) {
        discardPile.push_back(typed);
    }
}

template <typename T>
void CardDeck<T>::shuffle() {
    random_device rd;
    mt19937 rng(rd());
    std::shuffle(drawPile.begin(), drawPile.end(), rng);
}

template <typename T>
void CardDeck<T>::reshuffleDiscardPile() {
    drawPile.insert(drawPile.end(), discardPile.begin(), discardPile.end());
    discardPile.clear();
    shuffle();
}

template <typename T>
bool CardDeck<T>::isEmpty() const {
```

```
    return drawPile.empty() && discardPile.empty();
}

template <typename T>
int CardDeck<T>::drawPileSize() const {
    return static_cast<int>(drawPile.size());
}

template <typename T>
int CardDeck<T>::discardPileSize() const {
    return static_cast<int>(discardPile.size());
}
template class CardDeck<ChanceCard>;
template class CardDeck<CommunityChestCard>;
template class CardDeck<SkillCard>;
```

Pada baris terakhir template class CardDeck<ChanceCard>, template class CardDeck<CommunityChestCard>, dan template class CardDeck<SkillCard>. Inilah tiga instansiasi eksplisit dari satu template yang sama, menghasilkan tiga kelas yang sepenuhnya terpisah

4.4. Exception

Exception adalah mekanisme penanganan kesalahan yang terjadi saat runtime secara terstruktur. Exception dilempar menggunakan keyword “throw” ketika kondisi error terdeteksi, dan ditangkap menggunakan blok “try-catch” di lokasi yang mampu menanganinya.

4.4.1. Exception

```
#pragma once
#include <exception>
#include <string>

/// @brief Custom exception classes for handling various error scenarios in the Nimonspoli
game.
```

```
class GameException : public std::exception {
protected:
    std::string message;

public:
    GameException(const std::string& msg) : message(msg) {}
    const char* what() const noexcept override {
        return message.c_str();
    }
};

/// @brief Exception thrown when a player attempts to perform an action without sufficient
funds.
class InsufficientFundsException : public GameException {
public:
    InsufficientFundsException(int required, int available);
};

/// @brief Exception thrown when a player tries to purchase a property that is not
available for
/// sale.
class InvalidPropertyException : public GameException {
public:
    InvalidPropertyException(const std::string& reason);
};

/// @brief Exception thrown when a player tries to use a card that has reached its usage
limit.
class CardLimitException : public GameException {
public:
    CardLimitException();
};

/// @brief Exception thrown when a player attempts to perform an action that is not valid
in the
/// current game state.
```

```
class InvalidCommandException : public GameException {
public:
    InvalidCommandException(const std::string& command);
};

/// @brief Exception thrown when an error occurs during file operations, such as loading or
saving
/// game data.
class FileException : public GameException {
public:
    FileException(const std::string& filename, const std::string& operation);
};

/// @brief Exception thrown when a player tries to perform an action that is not valid in
the
/// current game state.
class GameStateException : public GameException {
public:
    GameStateException(const std::string& reason);
};

#include "Exceptions.hpp"
#include <iostream>

using namespace std;

InsufficientFundsException::InsufficientFundsException(int required, int available)
    : GameException("Uang kamu tidak cukup! Harga: M" + to_string(required) +
                    " | Uang kamu saat ini: M" + to_string(available)) {}

InvalidPropertyException::InvalidPropertyException(const string& reason) :
GameException(reason) {}

CardLimitException::CardLimitException()
    : GameException("PERINGATAN: Kamu sudah memiliki 3 kartu di tangan (Maksimal 3)! "
                    "Kamu diwajibkan membuang 1 kartu.") {}
```

```
InvalidCommandException::InvalidCommandException(const string& command) :
GameException(command) {}

FileException::FileException(const string& filename, const string& operation)
    : GameException([&]() -> string {
          if (operation == "simpan") {
              return "Gagal menyimpan file! Pastikan direktori dapat ditulis.";
          } else if (operation == "tidak_ditemukan") {
              return "File \"" + filename + "\" tidak ditemukan.";
          } else if (operation == "muat") {
              return "Gagal memuat file! File rusak atau format tidak dikenali.";
          }
          return "Kesalahan file \"" + filename + "\": " + operation;
      }()) {}

GameStateException::GameStateException(const string& reason) : GameException(reason) {}
```

4.4.2. Throwing Exception: Player::addCard()

Method addCard() pada Player melempar CardLimitException ketika pemain mencoba menyimpan lebih dari 3 kartu skill

```
void Player::addCard(SkillCard* card) {
    if (hand.size() >= 3) {
        throw CardLimitException();
    }
    hand.push_back(card);
}
```

4.4.3. Catching Exception di CommandHandler::dispatch()

CommandHandler menangkap seluruh GameException dalam satu blok catch dan meneruskan pesan error ke UI.

```
void CommandHandler::dispatch(const string& input, Player& player, IGameContext& ctx,
                              PropertyManager& propMgr, Board& board, Dice& dice,
                              CardDeck<SkillCard>* skillDeck, IUserInteraction* ui, Logger&
logger,
                              int& lastDiceTotal, SaveCallback saveCallback,
                              const vector<unique_ptr<Player>>& players) {
    if (input.empty())
        return;
    istringstream iss(input);
    string cmd;
    iss >> cmd;
    transform(cmd.begin(), cmd.end(), cmd.begin(), ::toupper);

    // Build raw pointer list for displayBoard
    vector<Player*> playerPtrs;
    for (const auto& p : players)
        playerPtrs.push_back(p.get());

    try {
        if (cmd == "CETAK_PAPAN") {
            if (ui)
                ui->displayBoard(board, playerPtrs, ctx.getCurrentTurn(),
ctx.getConfig().maxTurn);

        } else if (cmd == "LEMPAR_DADU") {
            if (player.getHasRolledDice()) {
                throw GameStateException("Kamu sudah melempar dadu pada giliran ini.");
            } else {
                handleLemparDadu(player, dice, ctx, logger, lastDiceTotal, ui);
            }

        } else if (cmd == "ATUR_DADU") {
```

```
            if (player.getHasRolledDice()) {
                throw GameStateException("Kamu sudah melempar dadu pada giliran ini.");
            } else {
                int d1 = -1, d2 = -1;
                if (iss >> d1 >> d2) {
                    handleLemparDadu(player, dice, ctx, logger, lastDiceTotal, ui, d1, d2);
                } else {
                    if (ui)
                        ui->printMessage("Format: ATUR_DADU X Y (contoh: ATUR_DADU 3
4)\n");
                }
            }

        } else if (cmd == "CETAK_AKTA") {
            string code;
            if (!(iss >> code)) {
                if (ui) {
                    ui->printMessage("Masukkan kode petak: ");
                    code = ui->readLine();
                }
            }
            Tile* t = board.getTileByCode(code);
            auto* prop = dynamic_cast<PropertyTile*>(t);
            if (prop) {
                if (ui)
                    ui->printPropertyDeed(*prop);
            } else {
                throw InvalidPropertyException("Petak \"" + code +
                                                "\" tidak ditemukan atau bukan properti.");
            }

        } else if (cmd == "CETAK_PROPERTI") {
            if (ui)
                ui->printPlayerProperties(player);

        } else if (cmd == "GADAI") {
```

```
            propMgr.mortgage(player, board, ctx.getCurrentTurn(), logger, ui);

        } else if (cmd == "TEBUS") {
            propMgr.redeem(player, ctx.getCurrentTurn(), logger, ui);

        } else if (cmd == "BANGUN") {
            propMgr.build(player, board, ctx.getCurrentTurn(), logger, ui);

        } else if (cmd == "GUNAKAN_KEMAMPUAN") {
            handleGunakanKemampuan(player, ctx, skillDeck, logger, ui);

        } else if (cmd == "CETAK_LOG") {
            int n = -1;
            iss >> n;
            auto logs = logger.getRecentLogs(n);
            if (ui)
                ui->printTransactionLogs(logs);

        } else if (cmd == "SIMPAN") {
            if (player.getHasRolledDice()) {
                throw GameStateException(
                    "SIMPAN hanya bisa dilakukan di awal giliran (sebelum melempar
dadu).");
            } else {
                string filename;
                if (!(iss >> filename))
                    filename = "nimonspoli_save.txt";

                if (filename.find("data/") != 0) {
                    filename = "data/" + filename;
                }
                // Check if file already exists — spec requires overwrite prompt
                {
                    ifstream checkFile(filename);
                    if (checkFile.good()) {
                        checkFile.close();
```

```
                        if (ui) {
                            ui->printMessage("File '" + filename +
                                             "' sudah ada. Timpa file lama? (y/n): ");
                            string response = ui->readLine();
                            if (response.empty() || (response[0] != 'y' && response[0] !=
'Y')) {
                                ui->printMessage("Penyimpanan dibatalkan.\n");
                                return;
                            }
                        }
                    }
                }
                saveCallback(filename);
                if (ui)
                    ui->printMessage("Permainan berhasil disimpan ke: " + filename + "\n");
                logger.logEvent(LogLevel::INFO, ctx.getCurrentTurn(), player.getUsername(),
                                "SIMPAN", "Simpan ke " + filename);
            }

        } else {
            throw InvalidCommandException(
                "Perintah tidak dikenali: " + cmd +
                ". Ketik CETAK_PAPAN, LEMPAR_DADU, ATUR_DADU X Y, CETAK_AKTA, "
                "CETAK_PROPERTI, GADAI, TEBUS, BANGUN, GUNAKAN_KEMAMPUAN, "
                "CETAK_LOG [N], SIMPAN [FILE].");
        }
    } catch (const GameException& e) {
        if (ui)
            ui->printMessage(string(e.what()) + "\n");
    }
}
```

4.5. C++ Standard Template Library

Standard Template Library (STL) adalah kumpulan class dan fungsi template yang telah dioptimalkan dan distandarisasi sebagai bagian dari C++ standard library.

4.5.1. std::vector

std::vector merupakan tipe STL yang merepresentasikan array dinamis pada C++. Seperti pada array biasa, pengaksesan elemen pada vektor juga menggunakan indeks. std::vector memungkinkan berbagai tipe data untuk digunakan sebagai array, sehingga penggunaannya tersebar luas pada proyek ini, seperti menyimpan kumpulan pemain, tile papan, kartu, dan log event. Penggunaan std::vector diperlukan karena jumlah elemen yang ada pada kumpulan data tersebut dapat berubah-ubah (dinamis) seiring keberjalanan program.

```
#pragma once
#include "ColorGroup.hpp"
#include <string>
#include <vector>


// Forward declarations — avoid pulling heavy headers into every translation unit
class PropertyTile;
class SkillCard;


enum class PlayerStatus {
    ACTIVE,
    BANKRUPT,
    JAILED
};


/// @brief Represents a player in the game.
class Player {
private:
    int id;
    std::string username;
    int money;
```

```
    PlayerStatus status;
    int position;
    int jailTurns;
    bool isComputer;


    std::vector<SkillCard*> hand;
    std::vector<PropertyTile*> properties;


    bool hasUsedSkillThisTurn;
    bool hasRolledDice;
    int consecutiveDoubles;


    // Active card-effect state
    int discountPercentage; ///< 0 = no active discount
    int discountTurnsLeft;
    int shieldTurnsLeft;   ///< 0 = not shielded
    bool getOutOfJailCard; ///< true if player has a "Bebas dari Penjara" card


public:
    Player(int id, const std::string& username);
    ~Player();


    // ── Operator overloads ──────────────────────────────────────────────
    Player& operator+=(int amount);
    Player& operator-=(int amount);
    /// @brief Compare by cash on hand.
    bool operator<(const Player& other) const;
    bool operator>(const Player& other) const;


    // ── Basic getters ───────────────────────────────────────────────────
```

```
    int getId() const;
    const std::string& getUsername() const;
    int getMoney() const;
    int getPosition() const;
    PlayerStatus getStatus() const;
    int getJailTurns() const;
    int getConsecutiveDoubles() const;
    const std::vector<SkillCard*>& getHand() const;
    const std::vector<PropertyTile*>& getProperties() const;


    bool getIsComputer() const;
    void setIsComputer(bool value);


    // ── Wealth helpers ──────────────────────────────────────────────────────
    /// @brief Cash on hand only.
    int getLiquidWealth() const;
    /// @brief Cash + full buy-price of all owned properties + building costs.
    int getTotalWealth() const;
    /// @brief Sum of full buy-prices of all owned properties (including mortgaged).
    int getPropertyValue() const;


    // ── Setters ─────────────────────────────────────────────────────────────
    void setPosition(int pos);
    void setStatus(PlayerStatus newStatus);
    void setJailTurns(int turns);
    void incrementJailTurns();
    void resetJailTurns();


    // ── Turn-state flags ────────────────────────────────────────────────────
    bool getHasUsedSkillThisTurn() const;
    void setUsedSkillThisTurn(bool value);
    bool getHasRolledDice() const;
```

```
    void setRolledDice(bool value);
    void resetTurnFlags();


    // ── Jail helpers ──────────────────────────────────────────────────────────
    bool isInJail() const;
    void goToJail();
    void releaseFromJail();


    // ── Property management ───────────────────────────────────────────────────
    void addProperty(PropertyTile* property);
    void removeProperty(PropertyTile* property);
    bool hasMonopoly(ColorGroup group) const;
    /// @brief Returns groups where the player owns all tiles.
    std::vector<ColorGroup> getMonopolyGroups() const;
```

4.5.2. std::unique_ptr

std::unique_ptr adalah kelas yang mengelola kepemilikan unik terhadap objek di heap. Artinya, hanya ada satu pemilik untuk setiap objek. Jika std::unique_ptr keluar dari scope atau dihapus, objek yang dikelolanya juga akan dihapus secara otomatis, mencegah memory leak. std::unique_ptr tidak dapat disalin, hanya bisa dipindahkan.

```
#pragma once


#include "../models/Players/Player.hpp"
#include "../models/Tiles/Tile.hpp"
#include <memory>
#include <vector>
```

```
/// @brief Manages the grid and specific tile positions of the Nimonspoli game board.
class Board {
private:
    /// @brief A collection of tiles representing the game board.
    std::vector<std::unique_ptr<Tile>> tiles;


public:
    /// @brief Creates the game board, optionally using dynamic board configuration.
    Board();


    /// @brief Destructor for the board.
    ~Board();


    /// @brief Gets the tile at the specified index or position.
    /// @param index The 0-based index of the tile on the board.
    /// @return A pointer to the requested tile.
    Tile* getTileAt(int index) const;


    /// @brief Gets a tile based on its 3-character code.
    /// @param code The unique code of the tile.
    /// @return A pointer to the requested tile, or nullptr if not found.
    Tile* getTileByCode(const std::string& code) const;


    /// @brief Calculates the new position of a player after moving a certain number
of steps.
    /// @param currentPosition The current index position of the player.
    /// @param steps The number of steps to move forward (or backward if negative).
    /// @return The new target position index on the board.
```

```
    int getNewPosition(int currentPosition, int steps) const;


    /// @brief Gets the total number of tiles on the board.
    /// @return The total tile count.
    int getTotalTiles() const;


    /// @brief Gets the index position of the Jail tile.
    /// @return The zero-based index of the jail block.
    int getJailPosition() const;


    /// @brief Appends a tile to the board (used during board construction).
    void addTile(std::unique_ptr<Tile> tile);
};
```

Pada kelas Board, seluruh tile papan dikelola menggunakan std::vector>. Karena Tile adalah abstract base class dengan banyak turunan polimorfis (StreetTile, CardTile, JailTile, dan lain-lain), penggunaan unique_ptr memungkinkan polimorfisme runtime berjalan dengan benar sekaligus menjamin tidak terjadi memory leak saat objek Board dihancurkan. Metode addTile() juga menerima unique_ptr secara langsung.

4.5.3. std::pair

std::pair adalah kelas template sederhana di C++ STL yang menyimpan dua objek berpasangan (Untuk dice, karena dice ada 2). Std::pair berguna untuk mengembalikan dua nilai dari sebuah fungsi tanpa perlu mendefinisikan struct baru, serta untuk mengelompokkan dua data yang berkaitan.

```
#pragma once
```

```
#include <random>
#include <string>
#include <vector>


/// @brief Simulates throwing two 6-sided dice for the game.
class Dice {
private:
    /// @brief Result of the first die.
    int die1;


    /// @brief Result of the second die.
    int die2;


    /// @brief Counter tracking how many consecutive times doubles have been rolled.
    int consecutiveDoubles;


    std::mt19937 rng;
    std::uniform_int_distribution<int> dist;


public:
    /// @brief Creates a dice object with an initial neutral state.
    Dice();
    ~Dice();


    /// @brief Randomizes the dice using an RNG, updating the die values, checking
doubles.
    /// @return Total value of the rolled dice.
```

```
    int rollRandom();


    /// @brief Sets the dice results manually by the user.
    /// @param d1 Result of the first die.
    /// @param d2 Result of the second die.
    /// @return Total value of the updated dice.
    int rollManual(int d1, int d2);


    /// @brief Gets the total accumulated result from the latest throw.
    /// @return Current total sum of die1 and die2.
    int getTotal() const;


    /// @brief Determines if the latest roll was a double (both die numbers are
equal).
    /// @return True if it was a double, false otherwise.
    bool isDouble() const;


    /// @brief Determines if the player has exceeded the speeding limit (throw
doubles 3 times in a row).
    /// @return True if the roll generated a speed violation (3 doubles).
    bool isSpeedingViolation() const;


    /// @brief Resets the continuous double rolls counter (for example, end of
player's turn).
    void resetDoublesCount();

    /// @brief Retrieves the individual results of the dice.
    /// @return A pair of integers containing the values of the two dice.
    std::pair<int, int> getDiceValues() const;
};
```

Pada kelas Dice, metode getDiceValues() mengembalikan std::pair yang berisi nilai dadu pertama dan dadu kedua secara bersamaan.

4.5.4. std::string

std::string adalah kelas untuk merepresentasikan urutan karakter secara dinamis.

```
#pragma once
#include <string>
#include <vector>


/// @brief Represents a single event that occurred in the game log.
class LogEntry {
private:
    int turnNumber;
    std::string username;
    std::string actionType;
    std::string details;


public:
    LogEntry(int turnNumber, const std::string& username, const std::string&
actionType,
             const std::string& details);


    /// @brief Gets the formatted string corresponding to the log entry specification
format.
    /// @return A formatted log entry string.
    std::string toString() const;


    /// @brief Gets the formatted string equivalent for saving.
```

```
    /// @return A formatted save entry string.
    std::string toSaveString() const;


    int getTurnNumber() const;
    std::string getUsername() const;
    std::string getActionType() const;
    std::string getDetails() const;
};


/// @brief Determines the severity level of a log entry.
enum class LogLevel {
    INFO,
    WARNING,
    ERROR
};


/// @brief Records and maintains all significant transaction events automatically in
a structured
/// format.
class Logger {
private:
    /// @brief Stored history of game events.
    std::vector<LogEntry> entries;


public:
    /// @brief Creates a new empty transaction log.
    Logger();
    ~Logger();


    /// @brief Appends a new event into the transaction history log.
```

```
    /// @param level The severity level of the log entry.
    /// @param turn The game cycle round when the event took place.
    /// @param player The username of the user performing the action.
    /// @param action The string identifier descriptor of the action type.
    /// @param details The narrative detailed summary of the event outcome.
    void logEvent(LogLevel level, int turn, const std::string& player, const
std::string& action,
                  const std::string& details);


    /// @brief Retrieves the recent N line prints from the internal log database.
    /// @param count The number of recent elements to print. Set to -1 to view all
histories.
    /// @return A collection of recent log strings describing events.
    std::vector<std::string> getRecentLogs(int count = -1) const;


    /// @brief Retrieves all entries formatting strings compatible for saving the
game state.
    /// @return A vector composed of serialized logs.
    std::vector<std::string> getSaveLogs() const;


    /// @brief Clears out the transaction history cache.
    void clear();
};
```

Pada kelas LogEntry, setiap entri log menyimpan nama pemain, tipe aksi, dan detail kejadian sebagai std::string. Kelas Logger kemudian menyimpan seluruh riwayat permainan dalam std::vector yang dapat diekspor sebagai std::vector untuk ditampilkan.

4.6. Konsep OOP lain

4.6.1. Abstract Base Class

Abstract Base Class (ABC) adalah kelas yang memiliki setidaknya satu pure virtual function sehingga tidak dapat diinstansiasi secara langsung. Kelas turunan diwajibkan mengimplementasikan seluruh pure virtual function tersebut sebelum dapat menjadi objek konkret.

```
#pragma once
#include <string>


// Forward declaration — avoids pulling in Player/Game headers here
class IGameContext;


/// @brief Abstract base class for all board tiles.
class Tile {
private:
    int id;
    std::string code;
    std::string name;


public:
    Tile(int id, const std::string& code, const std::string& name);
    virtual ~Tile();


    int getId() const;
    const std::string& getCode() const;
    const std::string& getName() const;
```

```
    /**
     * @brief Called when a player lands on this tile.
     * @param ctx Live game context — use to move players, charge money, trigger
events.
     */
    virtual void landedOn(IGameContext& ctx) = 0;
};
```

Kelas Tile adalah ABC untuk semua tile papan. Dengan mendeklarasikan landedOn() sebagai pure virtual (= 0), kelas Tile tidak dapat diinstansiasi secara langsung. Setiap jenis tile turunan seperti StreetTile, CardTile, JailTile, GoTile, dan sebagainya wajib mengimplementasikan landedOn() sesuai perilaku spesifik masing-masing tile tersebut.

5. Bonus Yang Dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. COM

COM adalah fitur yang memungkinkan pemain komputer bertindak sebagai lawan main. Ketika pemain mendaftarkan diri di awal permainan, setiap slot pemain dapat dikonfigurasi sebagai manusia atau COM. COM mampu melaksanakan seluruh aksi dasar permainan secara mandiri: melempar dadu, memutuskan pembelian properti, membayar sewa, mengelola gadai dan tebus properti, membangun rumah atau hotel, mengikuti lelang, menggunakan skill card. Setiap keputusan yang diambil COM ditampilkan di terminal dalam format `[NamaPemain (COM) | M] > PERINTAH`, sehingga pemain manusia dapat mengikuti jalannya giliran COM. Seluruh pilihan dari COM di *randomized*, kecuali pada beberapa *restriction*, misalnya pada auction dimana harus ada seseorang yang melakukan bid.

5.1.1.1. Implementasi COM

5.1.1.1.1. Player

isComputer ditambahkan sebagai member privat di kelas Player beserta getter dan setter-nya. Flag ini diinisialisasi false pada konstruktor dan di-set saat setup pemain berdasarkan input pengguna.

```
class Player {
private:
 // ...
 bool isComputer;
public:
 bool getIsComputer() const;
 void setIsComputer(bool value);
};
```

5.1.1.1.2. Registrasi COM

Di `GameView::promptPlayerSetup()`, setelah memasukkan nama, pengguna ditanya apakah slot tersebut adalah COM. Hasilnya diteruskan ke `Game::createGame()` yang memanggil `setIsComputer()` untuk tiap pemain.

```
Pilih (1/2/3): 1
Pilih papan (contoh: config-60) atau tekan enter untuk default (config):
Jumlah pemain (2-4): 2
Nama pemain 1: dap
Apakah 1 sebuah COM? (y/n) n
Nama pemain 2: yo
Apakah 2 sebuah COM? (y/n) y
```

```
vector<pair<string, bool>> GameView::promptPlayerSetup() {
    int count = 0;
    while (true) {
        cout << "Jumlah pemain (2-4): ";
        count = readInt();
```

```
        if (count >= 2 && count <= 4) {
            break;
        }
        cout << "Input tidak valid. Jumlah pemain harus antara 2 hingga
4.\n";
    }

    vector<pair<string, bool>> results(count);
    for (int i = 0; i < count; ++i) {
        string currentName;
        while (true) {
            cout << "Nama pemain " << (i + 1) << ": ";
            currentName = readLine();
            if (currentName.empty() || currentName.find_first_not_of("
\t\r\n") == string::npos) continue;

            istringstream iss(currentName);
            iss >> currentName;

            bool duplicate = false;
            for (int j = 0; j < i; ++j) {
                if (results[j].first == currentName) {
                    duplicate = true;
                    break;
                }
            }
            if (duplicate) {
                cout << "Nama " << currentName << " sudah digunakan.
Masukkan nama lain.\n";
            } else {
                break;
            }
        }

        bool isCom = false;
```

```
        while (true) {
            cout << "Apakah " << (i + 1) << " sebuah COM? (y/n) ";
            string ans = readLine();
            if (ans == "y" || ans == "Y") {
                isCom = true;
                break;
            } else if (ans == "n" || ans == "N") {
                isCom = false;
                break;
            } else {
                cout << "Input tidak valid. Masukkan y atau n.\n";
            }
        }
        results[i] = make_pair(currentName, isCom);
    }
    return results;
}
```

5.1.1.1.3. Giliran COM di Game::runTurn()

Pada Game::runTurn(), setiap iterasi command loop dicek apakah pemain aktif adalah COM. Jika ya, input ditentukan melalui kemungkinan, 20% kemungkinan GUNAKAN_KEMAMPUAN, 10% TEBUS, 10% BANGUN, dan sisanya LEMPAR_DADU.

```
[yo (COM) | M1000] > LEMPAR_DADU

Mengocok dadu...
Hasil: 2 + 2 = 4
Memajukan bidak yo sebanyak 4 petak...
Bidak mendarat di: Pajak Penghasilan.

Kamu mendarat di Pajak Penghasilan (PPH)!
Pilih opsi pembayaran pajak:
1. Bayar flat M150
2. Bayar 10% dari total kekayaan
Pilihan (1/2): 2
Total kekayaan kamu:
  Uang tunai          : M1000
  Harga beli properti : M0
  Total               : M1000
Pajak 10%             : M100
yo membayar M100 ke Bank. Uang: M900
Double! yo mendapat giliran tambahan.

[yo (COM) | M900] > LEMPAR_DADU

Mengocok dadu...
Hasil: 4 + 4 = 8
Memajukan bidak yo sebanyak 8 petak...
Bidak mendarat di: PLN.
PLN kini menjadi milik yo.
Double! yo mendapat giliran tambahan.

[yo (COM) | M900] > TEBUS
Tidak ada properti yang sedang digadaikan.

[yo (COM) | M900] > TEBUS
Tidak ada properti yang sedang digadaikan.
```

```
        if (player.getIsComputer()) {
            if (!player.getHasUsedSkillThisTurn() &&
!player.getHand().empty() &&
                rand() % 100 < 20) {
                input = "GUNAKAN_KEMAMPUAN";
            } else if (rand() % 100 < 10) {
                input = "TEBUS";
            } else if (rand() % 100 < 10) {
```

```
                    input = "BANGUN";
                } else {
                    input = "LEMPAR_DADU";
                }
                ui->printMessage("\n[" + player.getUsername() + " (COM) |
M" +
                                 to_string(player.getMoney()) + "] > " +
input + "\n");
            } else {
                input = ui->getCommandInput(player);
            }
            handleCommand(input, player);
        }
        return;
    }
```

5.1.1.1.4. Penanganan Penjara COM - handleJailTurn()

Ketika COM berada di penjara dan diminta memilih opsi keluar (BAYAR / LEMPAR / KARTU), keputusan diambil otomatis: 10% BAYAR jika uang mencukupi, 30% menggunakan kartu bebas penjara jika dimiliki, sisanya LEMPAR

```
   if (player.getIsComputer()) {
        int r = rand() % 100;
        if (r < 10 && player.getMoney() >= config.jailFine)
            choice = "BAYAR";
        else if (player.hasJailFreeCard() && r < 40)
            choice = "KARTU";
        else
            choice = "LEMPAR";
```

```
        ui->printMessage(choice + "\n");
    } else {
        choice = ui->readLine();
    }
```

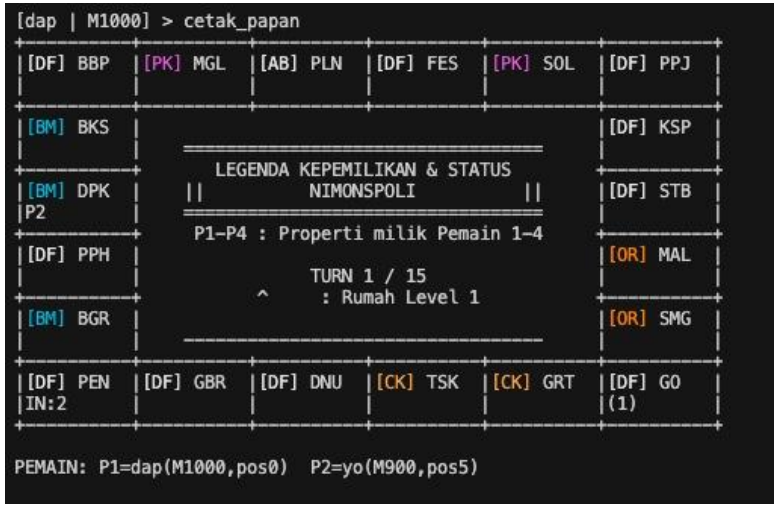
5.1.2. Papan Dinamis

Pemain dapat memilih direktori konfigurasi yang berbeda saat memulai permainan baru, misalnya config-60 untuk papan 60 petak. Folder config ini, misal config-60 harus ditaruh di root, setara dengan folder config default. Syarat yang harus dipenuhi oleh konfigurasi adalah: tepat 1 petak GO, tepat 1 petak Penjara (PEN), jumlah total petak antara 20 hingga 60, dan setiap properti yang digunakan harus ada di property.txt. Seluruh mekanisme permainan, sewa, lelang, festival, pajak, gadai, dan sebagainya tetap berjalan benar di atas konfigurasi papan apapun.

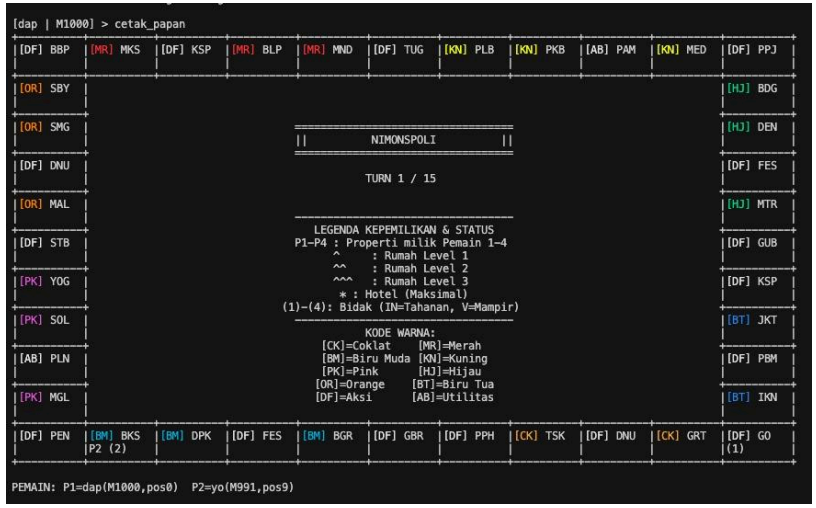
5.1.2.1. Implementasi Papan Dinamis

5.1.2.1.1. GameView::displayBoard()

Renderer papan di GameView::displayBoard() konstanta COLS = S, nilai S (sisi grid) dihitung dari totalTiles menggunakan rumus S = (totalTiles + 3) / 4 + 1. Fungsi pemetaan posisi getPos(row, col) juga berukuran S. Interior papan memiliki dua mode rendering: untuk papan standar 40 petak, untuk ukuran lain, legenda diisi sesuai slot yang tersedia.



Gambar papan dinamis ukuran 20



Gambar papan dinamis ukuran 40

Gambar papan dinamis ukuran 60

```
const int CW = 10;
const int S = (totalTiles + 3) / 4 + 1;
const int COLS = S;
const int INNER_W = (S - 2) * (CW + 1) - 1;

/***/

auto getPos = [S](int row, int col) -> int {
    if (row == S - 1)
        return (S - 1) - col;
    if (col == 0)
        return 2 * (S - 1) - row;
    if (row == 0)
        return 2 * (S - 1) + col;
    return 3 * (S - 1) + row;
};

/***/

const int numInteriorRows = S - 2;
const int contentSlots = numInteriorRows * 2;
const int sepSlots = max(0, numInteriorRows - 1);
vector<string> interior(contentSlots + sepSlots, pad("", INNER_W));

/***/

if (numInteriorRows == 9) {
    interior[0]  = pad("", INNER_W);
    interior[1]  = pad("", INNER_W);
    interior[2]  = centerStr(titleBox, INNER_W);
    interior[3]  = centerStr(titleTxt, INNER_W);
    interior[4]  = pad("", INNER_W);
    interior[5]  = centerStr(turnStr, INNER_W);
    /***/
} else {
    vector<string> legend;
    legend.push_back(pad("", INNER_W));
    legend.push_back(centerStr(titleBox, INNER_W));
```

```
    legend.push_back(centerStr(titleTxt, INNER_W));
    /***/
    for (int i = 0; i < static_cast<int>(interior.size()) &&
                    i < static_cast<int>(legend.size());
         ++i) {
        interior[i] = legend[i];
    }
}
```

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Papan Permainan	13524122, 13524133, 13524135, 13524139, 13524142	13524133
Petak Spesial	13524139, 13524142	13524135
Dadu dan Giliran Bermain	13524139, 13524142	13524122, 13524133
Kepemilikan Properti	13524122, 13524139	13524122
Ketentuan Sewa	13524122, 13524139	13524122
Gadai	13524139, 13524133	13524135
Peningkatan Properti	13524122, 13524139	13524122
Lelang Tanah	13524139, 13524142	13524135
Pajak	13524122, 13524139	13524122

Festival	13524122, 13524139	13524122
Kartu Kesempatan dan Dana Umum	13524139, 13524142	13524142
Kartu Kemampuan Spesial	13524139, 13524142	13524142
Bangkrut	13524139, 13524133	13524142
Pengambilalihan Aset	13524139, 13524142	13524142
Kondisi Game Selesai	13524139, 13524142	13524135
Save/Load	13524122, 13524139, 13524142	13524122
Logger	13524133, 13524139	13524135
Papan Dinamis	13524139	13524133, 13524139
COM	13524122	13524122, 13524133, 13524139

Lampiran

Lampiran A. Form Asistensi

Folder Asistensi